

Megaprompt — AGUSTÍN · Kernel, infraestructura, inventario y pagos

Pegar entero en Claude Code, parado en `~/Desktop/UAI/MDW-project/ecommerce-mdw`.

Sos el desarrollador a cargo del **kernel, la infraestructura, el inventario y los pagos** del e-commerce de BC Importados, el trabajo final de Metodologías y Desarrollo Web (UAI). Trabajás con otras tres personas: Yasmín (identidad y RBAC), Genaro (catálogo) y Nicolás (carrito, checkout y pedidos).

Hoy es domingo 04/10/2026. El Parcial I es el martes 06/10 y se defiende el backend corriendo en producción. La condición de aprobación es dura: si la URL pública no responde o el login no funciona, el parcial no se toma y pasa a la recuperación del 24/11. El repositorio hoy tiene la documentación hecha y prácticamente nada de código.

Tu bloque 0 es **bloqueante para las otras tres personas**: hasta que no esté pusheado, nadie puede compilar un handler. Hacelo primero, rápido y sin adornos.

Estado real del repositorio

Lo que ya existe y **no hay que reescribir**:

- `backend/config/viper.go` y su test: carga toda la configuración. **Ningún otro paquete puede leer variables de entorno** (es el ADR-001). Si necesitás una variable nueva, se agrega como campo con su tag `mapstructure` en `Config`, se valida ahí mismo y se documenta en el ADR.
- `backend/internal/adapters/rest/router/router.go`: devuelve un `chi.Mux` vacío.
- `backend/internal/adapters/db/postgresql/migrations/migrator.go`: `NewMigrator(db, migrations)`.
- `docs/specs.md` (553 líneas), `docs/erd.dbml`, `docs/adr/ADR-001..003`, `docs/rfc/RFC-001..006`.
- `go.mod` con `chi`, `gormigrate`, `viper` y `gorm`. **Falta el driver de Postgres**.

Lo que no existe: `cmd/api/main.go`, entidades, migraciones, repositorios, handlers, CI (el workflow está vacío) y la URL pública.

Decisiones ya tomadas que tenés que respetar

- Go 1.26 · Chi · GORM · gormigrate · PostgreSQL. Está en los ADR-001, 002 y 003.
- **Plata en `int64` de centavos**, columnas `bigint`, campos `..._centavos`. Nunca `float64`.
- **Identificadores `uuid.UUID`**, columnas `uuid`. Nunca autoincremental.
- Nombres de tablas y columnas en castellano y `snake_case`, como el ERD.
- Capas: el handler traduce HTTP y nada más; las reglas viven en `usecases`; las consultas en `repositories`. Un handler no arma un `db.Where(...)`.

Reglas duras

1. **Sólo tocás los archivos que esta consigna te asigna**. No edites archivos de Yasmín, Genaro ni Nicolás, ni siquiera para "arreglar" algo. Si algo de ellos te bloquea, dejá un `TODO` y seguí.
2. `go build ./...` y `go vet ./...` en verde antes de cada push. Si algo no compila por culpa de un módulo ajeno que todavía no existe, comentá el mount y dejalo anotado.

3. La identidad baja del contexto de la sesión. Nunca de un body, un header propio o la query.
4. Ningún secreto en el repo. `.env` va al `.gitignore`; se commitea `.env.example` con los nombres y valores vacíos.
5. Cada commit en español, en imperativo, y es una pieza que compila. Al terminar cada bloque: `git add` de los archivos de ese bloque, `git commit`, `git push`. **No acumules trabajo sin pushear**: la nota es individual y se mide sobre el historial del repositorio.
6. Trabajás sobre las ramas que indica cada bloque. `main` está protegida.

BLOQUE 0 — El kernel (BLOQUEANTE, máximo 2 horas)

Rama: `feat/kernel`, salida de `main`. Antes de nada creá también `develop`:

```
git fetch --all
git checkout main && git pull
git checkout -b develop && git push -u origin develop
git checkout -b feat/kernel
```

0.1 Dependencias (sos el único que toca `go.mod`)

Agregá de una sola vez todas las que el equipo va a necesitar, para que nadie más tenga que tocar `go.mod` y no haya conflictos:

```
cd backend
go get gorm.io/driver/postgres
go get github.com/google/uuid
go get github.com/golang-jwt/jwt/v5
go get golang.org/x/crypto/bcrypt
go get github.com/go-playground/validator/v10
go get github.com/go-chi/cors
go mod tidy
```

Mercado Pago se integra con `net/http` contra su API REST, sin SDK: menos dependencias y más fácil de defender.

0.2 Configuración

En `backend/config/viper.go` agregá los campos que faltan, respetando el estilo del archivo y documentándolos después en el ADR-001:

- `DatabaseDSN` ya existe. Verificá que sea requerida.
- `MercadoPagoAccessToken`, `MercadoPagoWebhookSecret` ya existen.
- Agregá `PublicBaseURL` (`PUBLIC_BASE_URL`, requerida): la URL pública del backend, que Mercado Pago necesita para las `back_urls` y el webhook.

0.3 Errores HTTP — `backend/internal/adapters/rest/errors/errores.go`

Paquete `apierr`. Es el contrato de errores de todo el sistema y lo van a importar los cuatro:

```

package apierr

type Cuerpo struct {
    Error    string `json:"error"`
    Detalles any    `json:"detalles,omitempty"`
}

var (
    ErrNoAutenticado = errors.New("no autenticado")
    ErrNoAutorizado  = errors.New("no autorizado")
    ErrNoEncontrado  = errors.New("no encontrado")
)

type ErrValidacion struct{ Campos map[string]string }
func (e ErrValidacion) Error() string { ... }

type ErrRegla struct {
    Mensaje string
    Datos   any
}
func (e ErrRegla) Error() string { return e.Mensaje }

func JSON(w http.ResponseWriter, status int, cuerpo any)
func Responder(w http.ResponseWriter, endpoint string, err error)

```

`Responder` es la única traducción de error a status del proyecto:

Error	Status	Cuerpo
<code>ErrNoAutenticado</code>	401	<code>{"error":"No autenticado"}</code>
<code>ErrNoAutorizado</code>	403	<code>{"error":"No podés realizar esta operación"}</code>
<code>ErrNoEncontrado</code>	404	<code>{"error":"No encontrado"}</code>
<code>ErrValidacion</code>	400	<code>{"error":"Datos inválidos","detalles":{"campos":{...}}}</code>
<code>ErrRegla</code>	409	<code>{"error":<Mensaje>,"detalles":<Datos>}</code>
cualquier otro	500	<code>{"error":"Error interno"}</code> y <code>log.Printf("%s: %v", endpoint, err)</code>

El detalle del 500 va al log del servidor, que es nuestro; la respuesta la lee un desconocido. Escribí `errores_test.go` con un caso por fila de esa tabla.

0.4 Conexión y migrador

`backend/internal/adapters/db/postgresql/conexion.go`:

```

func Abrir(dsn string) (*gorm.DB, error)

```

Abre con `postgres.Open(dsn)`, configura el pool (`SetMaxOpenConns(10)`, `SetMaxIdleConns(5)`, `SetConnMaxLifetime(time.Hour)`) y hace un `Ping`.

`backend/internal/adapters/db/postgresql/migrations/registry.go`:

```
func Todas() []*gormigrate.Migration
```

Concatena, **en este orden fijo**, las migraciones de los cuatro módulos:

```
schema.MigracionesIdentidad() // Yasmín 20261004_01_*
schema.MigracionesCatalogo()  // Genaro 20261004_02_*
schema.MigracionesInventario() // vos 20261004_03_*
schema.MigracionesPedidos()   // Nicolás 20261004_04_*
```

El orden respeta las foreign keys. Como esos archivos todavía no existen, creá en `migrations/schema/` un archivo **por módulo** con la función devolviendo un slice vacío y un comentario `// Lo completa <persona>`. Cada dueño después reemplaza el contenido de **su** archivo. Así el proyecto compila desde el minuto cero y nadie se pisa.

0.5 Punto de entrada — `backend/cmd/api/main.go`

```
func main() {
    cfg, err := config.Load() // única lectura de entorno
    db, err := postgresql.Abrir(cfg.DatabaseDSN)

    if len(os.Args) > 1 && os.Args[1] == "migrate" {
        // corre gormigrate con migrations.Todas() y termina
    }

    mw := middleware.Nuevo(cfg.AppSecretKey) // de Yasmín
    r := router.NewRouter()
    r.Use(chimw.RequestID, chimw.RealIP, chimw.Recoverer, chimw.Logger)
    r.Use(cors.Handler(...)) // permitir el origen del frontend
    r.Get("/api/salud", salud)

    identidad.Montar(r, db, cfg, mw)
    catalogo.Montar(r, db, cfg, mw)
    inventario.Montar(r, db, cfg, mw)
    pedidos.Montar(r, db, cfg, mw)

    http.ListenAndServe(":"+cfg.AppPort, r)
}
```

Las migraciones **no** corren al arrancar el servidor: es un subcomando explícito (`./api migrate`). Lo dice el ADR-002 y es de las cosas que se preguntan.

Mismo truco que con las migraciones: creá en `backend/internal/adapters/rest/handlers/<módulo>/rutas.go` un archivo por módulo con `func Montar(r chi.Router, db *gorm.DB, cfg config.Config, mw *middleware.Middleware) {}` vacío y el comentario de quién lo completa. Idem `middleware.Nuevo` de Yasmín: si todavía no existe, creá un stub mínimo en `middleware/middleware.go` con la firma del contrato que devuelve 401 siempre, y avísale que lo reemplace. **Ese archivo pasa a ser de ella en cuanto lo toque.**

`GET /api/salud` devuelve `200 {"estado":"ok","version":"<git sha corto>"}`. Es lo primero que vamos a mirar el martes.

0.6 Entorno y arranque local

- `backend/.env.example` con todas las variables de `Config`, con valores vacíos.
- `backend/Makefile`:

```
run:      set -a; . ./env; set +a; go run ./cmd/api
migrate:  set -a; . ./env; set +a; go run ./cmd/api migrate
test:     go test ./...
check:    go build ./... && go vet ./... && go test ./...
```

- Agregá `backend/.env` y `docs/.env` al `.gitignore`.

0.7 Base de datos en la nube

Creá el proyecto de Postgres (Supabase, región São Paulo, o el Postgres de la VPS). Guardá el DSN, pasáelo a los otros tres **por privado, nunca por el repo**, y corré `make migrate` para dejar el esquema vacío inicializado.

Una sola persona corre las migraciones contra la base compartida: vos. Si los cuatro corren `migrate` van a chocar.

0.8 ADR-004 y ADR-005

- `docs/adr/ADR-004-tipos-de-datos.md`: por qué los importes se guardan como `int64` de centavos y no como `float` (no representa exacto 0,10 y los errores se acumulan al sumar) ni como `decimal` (una dependencia más para la misma garantía), y por qué los identificadores son UUID y no autoincrementales (un autoincremental revela cuántos pedidos hay y deja adivinar el del vecino).
- `docs/adr/ADR-005-despliegue.md`: VPS propia en lugar de Vercel. Qué ganamos (control del runtime de Go, una sola máquina para API y base) y qué resignamos (no hay preview por Pull Request, que la rúbrica puntúa como destacado en Producción; el deploy lo dispara CI por push a `main`).
- Actualizó la tabla de `docs/adr/README.md`.

Commit y push del bloque 0

```
cd ..
git add backend/go.mod backend/go.sum backend/config/viper.go \
      backend/internal/adapters/rest/errors/ \
      backend/internal/adapters/db/postgresql/ \
      backend/internal/adapters/rest/handlers/ \
      backend/internal/adapters/rest/middleware/ \
      backend/cmd/ backend/Makefile backend/.env.example .gitignore \
      docs/adr/
git commit -m "feat(kernel): punto de entrada, conexion, migrador, contrato de errores y ADR-004/005"
git push -u origin feat/kernel
gh pr create --base develop --title "Kernel del backend" --body "Punto de entrada, conexión a
Postgres, migrador versionado, contrato de errores y stubs de los cuatro módulos. Desbloquea al resto
del equipo."
```

Avisá al grupo en cuanto esté mergeado. Yasmín, Genaro y Nicolás están esperando esto.

BLOQUE 1 — Dominio e inventario

Rama: `feat/inventario`, salida de `develop` actualizado.

1.1 Entidades — `backend/internal/core/domain/inventario.go`

Del ERD, recortado a lo que el Parcial I necesita:

- `Lote`: `id`, `variante_id`, `proveedor_id`, `unidades_ingresadas`, `costo_unitario_centavos`, `fecha_ingreso`, `creado_en`.
- `Proveedor`: `id`, `nombre` (obligatorio), `correo`, `telefono`, `web`, `creado_en`.
- `MovimientoStock`: `id`, `lote_id`, `tipo` (`INGRESO` / `AJUSTE` / `SALIDA`), `unidades`, `motivo`, `responsable_id` (nullable), `pedido_id` (nullable), `creado_en`.
- `ReservaStock`: `id`, `pedido_id`, `lote_id`, `unidades`, `estado` (`ACTIVA` / `CONSUMIDA` / `LIBERADA`), `vence_en` (nullable), `creado_en`.

Los movimientos y las reservas **no se borran ni se editan**: son historia. Un error se corrige con un movimiento compensatorio.

1.2 Migración — `migrations/schema/inventario.go`

Reemplazá tu stub. Una sola migración con ID `20261004_03_inventario`.

Índices obligatorios, porque son foreign keys por las que se filtra en cada pantalla: `lote(variante_id)`, `lote(proveedor_id)`, `movimiento_stock(lote_id)`, `reserva_stock(pedido_id)`, `reserva_stock(lote_id, estado)`.

`onDelete` decidido explícitamente en cada FK: **RESTRICT en todas**. Borrar un proveedor no puede llevarse puestos sus lotes, y borrar un lote no puede llevarse su historial de movimientos. Dejá el motivo escrito como comentario arriba de cada constraint: eso es exactamente lo que se pregunta.

```
git add backend/internal/core/domain/inventario.go \
        backend/internal/adapters/db/postgresql/migrations/schema/inventario.go
git commit -m "feat(inventario): entidades de lote, proveedor, movimiento y reserva con su migracion"
git push -u origin feat/inventario
```

1.3 Repositorios

`backend/internal/adapters/db/postgresql/repositories/inventario/`:

- `lotes.go`: `Crear`, `PorVariante(varianteID)` []`Lote` ordenado por `fecha_ingreso` ascendente (FIFO) o descendente (LIFO), `PorID`.
- `movimientos.go`: `Crear`, `PorVariante(varianteID, limit, offset)`.
- `reservas.go`: `Crear`, `ActivasPorPedido`, `ActivasPorLote`, `CambiarEstado`.
- `proveedores.go`: `Crear`, `Listar`, `PorID`.

Todo listado lleva `Limit` y `Offset` explícitos. Nunca un `Find` sin límite: el día que haya 40.000 filas ese endpoint las trae todas.

```
git add backend/internal/adapters/db/postgresql/repositories/inventario/
git commit -m "feat(inventario): repositorios de lotes, movimientos, reservas y proveedores"
git push
```

1.4 La regla pura de disponibilidad

`backend/internal/usecases/inventario/disponibilidad.go`. Sin importar GORM, sin importar `net/http` y sin llamar a `time.Now()` adentro: todo entra por parámetro. Así se puede testear llamándola.

```
type LoteDisponible struct {
    LoteID      uuid.UUID
    FechaIngreso time.Time
    Unidades    int    // ingresadas + ajustes - salidas
    Reservadas  int    // reservas ACTIVA sobre ese lote
}

// Unidades realmente vendibles.
func Disponible(lotes []LoteDisponible) int

// Qué lotes y cuántas unidades de cada uno cubren el pedido, según la metodología del producto.
// Devuelve error si no alcanza.
func AsignarLotes(lotes []LoteDisponible, unidades int, metodologia string) ([]Asignacion, error)
```

Regla de la spec: *stock disponible = existencias de lotes – reservas activas*. FIFO asigna primero los lotes ingresados antes; LIFO, los más recientes. Una compra puede consumir varios lotes.

`disponibilidad_test.go`, mínimo cinco casos: alcanza con un lote; hace falta partir entre dos lotes; no alcanza; hay unidades pero están todas reservadas; LIFO devuelve otro orden que FIFO. El cuarto es el que suele faltar y el que se pregunta.

```
git add backend/internal/usecases/inventario/
git commit -m "feat(inventario): regla pura de disponibilidad y asignacion FIFO/LIFO con tests"
git push
```

1.5 El servicio de stock (lo consume Nicolás)

`backend/internal/usecases/inventario/servicio_stock.go`. Esta firma está congelada: Nicolás ya está programando contra ella. No la cambies sin avisarle.

```

type ItemReserva struct {
    VarianteID uuid.UUID
    Unidades    int
}

func NuevoServicioStock(db *gorm.DB) *ServicioStock

func (s *ServicioStock) Disponible(ctx context.Context, varianteID uuid.UUID) (int, error)
func (s *ServicioStock) Reservar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID, items
[]ItemReserva) error
func (s *ServicioStock) Confirmar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID) error
func (s *ServicioStock) Liberar(ctx context.Context, tx *gorm.DB, pedidoID uuid.UUID) error

```

- **Reservar** recibe la transacción de quien llama y crea filas en `reserva_stock` con estado `ACTIVA`. Si falta stock devuelve `apierr.ErrRegla{Mensaje: "No hay stock suficiente", Datos: map[string]int{<varianteID>: <disponible>}}`. Ese mapa es lo que el checkout le enumera al comprador, y es un criterio de aceptación, no un adorno.
- Para evitar que dos checkouts simultáneos reserven la última unidad, lee los lotes con `clause.Locking{Strength: "UPDATE"}` dentro de la transacción.
- **Confirmar** pasa las reservas del pedido a `CONSUMIDA` y genera un `MovimientoStock` de tipo `SALIDA` por cada una. Es **idempotente**: si ya están consumidas, no hace nada y no devuelve error. Una notificación repetida de Mercado Pago no puede descontar dos veces.
- **Liberar** pasa las `ACTIVA` a `LIBERADA`, también idempotente, y **no** genera movimiento: liberar una reserva no es una entrada física.

```

git add backend/internal/usecases/inventario/servicio_stock.go
git commit -m "feat(inventario): servicio de reserva, confirmacion y liberacion de stock idempotente"
git push

```

Avisale a Nicolás apenas esto esté en `develOp`.

1.6 Endpoints de administración de stock

`backend/internal/adapters/rest/handlers/inventario/rutas.go`. Todos bajo `mw.RequierePermiso("stock.gestionar")`.

Método y ruta	Qué hace	Errores
POST <code>/api/admin/proveedores</code>	Alta de proveedor	400, 401, 403
GET <code>/api/admin/proveedores</code>	Listado paginado	401, 403
POST <code>/api/admin/lotes</code>	Ingreso de lote: variante, proveedor, unidades, costo unitario, fecha. Genera <code>MovimientoStock</code> de tipo <code>INGRESO</code>	400, 401, 403, 404
GET <code>/api/admin/variantes/{id}/stock</code>	Disponible y detalle por lote	401, 403, 404
GET <code>/api/admin/variantes/{id}/movimientos</code>	Historial paginado	401, 403, 404

Método y ruta	Qué hace	Errores
POST /api/admin/variantes/{id}/ajustes	Ajuste con unidades, motivo y responsable de la sesión. No puede dejar stock negativo ni consumir unidades reservadas en otro pedido	400, 401, 403, 409

El orden adentro de cada handler, siempre el mismo:

1. sesión y permiso → 401 / 403 (el middleware ya lo hizo; el responsable sale del contexto)
2. pertenencia → 404 (va en el WHERE de la consulta, no en un if)
3. validar el body → 400 (validator, y nunca `data: body`)
4. regla de negocio → 409 (la función pura)
5. hacer el trabajo

El `responsable_id` de un ajuste sale de `middleware.UsuarioDeContexto`, **nunca del body**. Es la regla que hace que la auditoría sirva para algo.

Un `defer / recover` no: un solo `defer func(){ ... }` no hace falta, pero sí `defer apierr.Responder(...)` no aplica — usá el patrón de un `err` acumulado y una sola llamada a `apierr.Responder(w, "POST /api/admin/lotess", err)` al final del camino de error.

```
git add backend/internal/adapters/rest/handlers/inventario/
git commit -m "feat(inventario): endpoints de proveedores, lotes, movimientos y ajustes"
git push
```

1.7 Datos de prueba

`backend/cmd/api/main.go` no se toca. Creá `backend/cmd/seed/main.go`, un comando aparte:

- 1 proveedor.
- Lotes para las variantes que siembre Genaro: al menos **una variante con stock suficiente, una con una sola unidad y una con cero**. Si todo tiene stock de sobra, la mitad de las reglas de negocio no se puede probar y la demo del martes no muestra nada.
- Idempotente: se corre con `FirstOrCreate` y se puede ejecutar muchas veces sin duplicar.

```
git add backend/cmd/seed/
git commit -m "feat(inventario): comando de seed con lotes que permiten probar las reglas de stock"
git push
```

BLOQUE 2 — Mercado Pago (el servicio externo del núcleo obligatorio)

El núcleo obligatorio de la materia pide una integración externa y la rúbrica la puntúa adentro del criterio de API. Destacado es "maneja la falla del proveedor y la comunica al usuario", así que el manejo de error no es opcional.

2.1 Adaptador

`backend/internal/adapters/external/mp/cliente.go`, con `net/http` y `MERCADOPAGO_ACCESS_TOKEN` de `sandbox`:

```
func NuevoCliente(accessToken, baseUrl string) *Cliente

func (c *Cliente) CrearPreferencia(ctx context.Context, p Preferencia) (RespuestaPreferencia, error)
func (c *Cliente) ObtenerPago(ctx context.Context, pagoID string) (Pago, error)
```

- `POST https://api.mercadopago.com/checkout/preferences` con los ítems, el `external_reference` = id del pedido, las `back_urls` armadas con `cfg.PublicBaseURL` y `notification_url` apuntando al webhook.
- Timeout de 10 segundos en el `http.Client`. Si Mercado Pago no responde, el error que sube es `apierr.ErrRegla{Mensaje: "No pudimos iniciar el pago, probá de nuevo en unos minutos"} → 409`, y **el pedido no queda marcado como pagado**. Eso es lo que la rúbrica llama manejar la falla del proveedor.

2.2 Entidad y migración

`IntentoPago`: `id`, `pedido_id`, `proveedor` (`MERCADO_PAGO / EFECTIVO`), `referencia_externa`, `estado` (`PENDIENTE / APROBADO / RECHAZADO`), `monto_centavos`, `creado_en`, `actualizado_en`. Índice único en (`proveedor`, `referencia_externa`) — es lo que hace imposible procesar dos veces la misma notificación.

Migración nueva con ID `20261004_03b_pagos`. **No edites la migración de inventario, que ya está mergeada.**

2.3 Endpoints

Método y ruta	Quién	Qué hace
<code>POST /api/pedidos/{id}/pago</code>	comprador (sesión opcional, con token de invitado)	Crea el intento y devuelve <code>init_point</code>
<code>POST /api/pagos/webhook</code>	público a propósito	Recibe la notificación de Mercado Pago
<code>GET /api/pedidos/{id}/pago</code>	comprador	Estado del último intento

El webhook es el endpoint más importante del módulo y el que se va a mirar en la defensa:

1. Verifica la firma con `MERCADOPAGO_WEBHOOK_SECRET`. Sin firma válida, 401 y no se procesa nada.
2. **Nunca confía en el cuerpo de la notificación para saber si está pagado.** Lee el `payment_id` y llama a `ObtenerPago` para preguntarle al proveedor. Una URL de retorno del navegador no confirma un pago.
3. Si el pago está aprobado: abre una transacción, marca el intento `APROBADO`, llama a `servicioStock.Confirmar(ctx, tx, pedidoID)` y deja el pedido confirmado.
4. **Idempotente**: la misma notificación dos veces no duplica movimientos, ni cobros, ni confirmaciones. El índice único de `referencia_externa` más un chequeo de estado lo garantizan.
5. Siempre responde 200 a Mercado Pago, salvo firma inválida: si devolvés 500 te reintenta en loop.

Escribí `webhook_test.go` con el caso que importa: **la misma notificación procesada dos veces deja el sistema igual que una sola vez.**

```
git add backend/internal/adapters/external/mp/ \
    backend/internal/core/domain/pagos.go \
    backend/internal/adapters/db/postgresql/migrations/schema/inventario.go \
    backend/internal/adapters/rest/handlers/inventario/
git commit -m "feat(pagos): integracion con Mercado Pago sandbox y webhook idempotente verificado por firma"
git push
```

BLOQUE 3 — Producción (es la condición de aprobación del parcial)

Esto va antes que cualquier funcionalidad que falte. Sin URL pública no hay parcial.

3.1 Contenedor

`backend/Dockerfile`, multi-stage: `golang:1.26-alpine` para compilar con `CGO_ENABLED=0 go build -o /api` `./cmd/api`, y `alpine` para correr. `EXPOSE 8080`, usuario no root.

`docker-compose.yml` en la raíz: el servicio `api` y, si la base va en la misma VPS, `postgres:16` con volumen persistente. Las variables salen del entorno, nunca hardcodeadas.

3.2 Despliegue en la VPS

- Nginx como reverse proxy con TLS (Let's Encrypt) apuntando al contenedor.
- **La URL no cambia en todo el cuatrimestre.** Pegala en el `readme.md`, en la sección Producción.
- Corré las migraciones contra la base de producción con `./api migrate` **antes** de levantar el servidor, no al arrancar.
- Cargá todas las variables de entorno en la VPS. El síntoma de que falta alguna es que en local anda y en producción no entra nadie.

Verificación: `curl https://<la-url>/api/salud` tiene que devolver 200 desde una máquina que no es la tuya. Probalo desde el celular con los datos móviles.

3.3 CI/CD — `.github/workflows/cicd.yaml` (hoy está vacío)

Dos jobs:

- `verificar`, en cada push y cada PR: `go build ./...`, `go vet ./...`, `go test ./...` y `gofmt -l .` (que no devuelva nada).
- `desplegar`, sólo en push a `main`: SSH a la VPS, `git pull`, `docker compose build`, `./api migrate`, `docker compose up -d`. Las credenciales van en GitHub Secrets.

```
git add Dockerfile docker-compose.yml .github/workflows/cicd.yaml readme.md
git commit -m "feat(infra): contenedor, despliegue en VPS y pipeline de verificacion y deploy"
git push
```

3.4 Pruebas de la API sin interfaz

`docs/api/inventario.http`, con `@base` apuntando primero a `http://localhost:8080` y la línea de producción comentada al lado. Por cada endpoint tuyo, **tres requests**: la que anda, la que manda un body inválido (tiene que dar 400, no 500) y la de un recurso que no existe (404). Más las tres de autorización: sin cookie → 401, con el rol equivocado → 403, con un recurso ajeno → 404.

Arriba de cada request escribí el status esperado. Mirá el número, no el cuerpo: un 500 donde iba un 409 se ve igual de roto desde afuera y sólo el status los distingue.

Después del merge, descomentá la línea de producción y corré el archivo entero otra vez. **Un endpoint que anda en localhost y no en la URL pública, para esta materia, no existe.**

```
git add docs/api/inventario.http
git commit -m "docs(inventario): requests de prueba con los tres casos por endpoint"
git push
```

BLOQUE 4 — Cierre

1. Recortá `docs/specs.md`, sección 9: pasá a *Fuera de alcance de esta entrega* todo lo que no entra en el Parcial I — promociones, cupones, facturación con ARCA, estadísticas, favoritos, avisos de reposición, ventas externas, auditoría, WhatsApp, correo y geocodificación con API de mapas. Lo que no está recortado por escrito, en la defensa cuenta como prometido y no entregado.
2. `docs/adr/ADR-006-sesion.md`, dos párrafos: los permisos viajan en el JWT, no se consulta la base en cada request. Qué ganamos (cero consultas por request) y qué resignamos (si el dueño le cambia el rol a alguien, recién aplica cuando esa persona vuelve a entrar; no se puede revocar al instante). Coordinalo con Yasmín, que es la dueña del middleware.
3. Actualizá el `readme.md`: URL de producción, cómo levantar el backend, cómo correr migraciones y seed, y la tabla de ADRs.

```
git add docs/ readme.md
git commit -m "docs: recorte de alcance para el Parcial I, ADR-006 y readme con la URL de produccion"
git push
gh pr create --base develop --title "Inventario, pagos e infraestructura" --body "Reservas de stock FIFO/LIFO, Mercado Pago sandbox con webhook idempotente, despliegue en VPS y CI."
```

Definición de terminado

No te detengas hasta que todo esto sea cierto:

- ☐ `make check` en verde: build, vet y tests.
- ☐ `curl https://<url-publica>/api/salud` devuelve 200 desde afuera de tu máquina.
- ☐ Las migraciones de los cuatro módulos aplicadas en la base de la nube, y
- ☐ El seed deja al menos una variante con stock, una con una unidad y una con cero.
- ☐ Un checkout que pide más unidades de las que hay devuelve **409 enumerando qué falta**, no un

- ☐ La misma notificación de Mercado Pago procesada dos veces deja el sistema idéntico.
- ☐ `docs/api/inventario.http` corre entero contra producción.
- ☐ Todo pusheado, con commits repartidos y a tu nombre.

Si te trabás

- Si algo no compila por un módulo que todavía no existe, comentá ese mount en `main.go`, dejá el `TODO` con el nombre de quien lo debe, seguí con lo tuyo y avisá por el grupo.
- Si una migración falla, **nunca** edites una migración ya aplicada ni corras un reset contra la base de la nube. Se corrige con una migración nueva.
- Si algo del alcance no llega, es preferible un módulo menos que un endpoint que devuelve 500. Recortá, anotalo en la spec y decilo en la defensa: eso suma, un bug escondido resta.

Lo que te van a preguntar el martes

Tenés que poder contestar estas sin mirar el código:

1. Mostrame qué pasa si pido esto con la cuenta de otro.
2. ¿Por qué el stock está en `int64` y los importes en centavos y no en `float` ?
3. ¿Qué pasa si Mercado Pago manda la misma notificación dos veces? ¿Y si llega tarde, después de que se liberó la reserva?
4. ¿Por qué las migraciones no corren al arrancar el servidor?
5. ¿Por qué `RESTRICT` y no `CASCADE` en los lotes?
6. ¿Por qué su app es un monolito y no una arquitectura de servicios?